UNIVERSITÉ DU QUÉBEC EN OUTAOUAIS

INF4173 - PROJET DE SYNTHÈSE

PRÉSENTÉ À

MYRIA [redacted]

BACCALARAURÉAT EN INFORMATIQUE

CONCENTRATION CYBER SÉCURITÉ

PAR

LUCAS [redacted]

whiteRabbit - Une application de messagerie iOS, résistante post-quantique

https://github.com/Loux14/whiteRabbit

1. Introduction 6
1.1 Contexte 6
1.2 La menace “Harvest Now, Decrypt Later” 6
1.3 Standardisation post-quantique 6
1.4 Problématique 7
1.5 Objectifs 7
2. État de l’art 8
2.1 La menace quantique sur la cryptographie actuelle 8
2.1.1 L’algorithme de Shor 8
2.1.2 L’algorithme de Grover 8
2.1.3 Ce qui doit être remplacé 8
2.1.4 Déjà cassés classiquement 9
2.2 Standards NIST post-quantiques 9
2.2.1 ML-KEM (FIPS 203) 9
2.2.2 ML-DSA (FIPS 204) 9
2.3 Protocoles de messagerie sécurisée existants 10
2.3.1 Le protocole Signal 10
2.3.2 PQXDH 10
2.3.3 Apple PQ3 10
2.3.4 Fonctionnement hybride — illustration 10
2.3.5 Comparatif des applications existantes 11
2.3.6 Lacunes identifiées 11

2.4 Librairies iOS disponibles 12
3. Architecture et protocole 13
3.1 Les algorithmes cryptographiques utilisés 13
3.1.1 Ed25519 — signatures classiques 13
3.1.2 X25519 — échange de clés classique 13
3.1.3 AES-256-GCM — chiffrement symétrique 14
3.1.4 HKDF-SHA256 — dérivation de clés 14
3.1.5 ML-KEM-768 — encapsulation post-quantique 14
3.1.6 ML-DSA-65 — signatures post-quantiques 15
3.2 Types de clés et leurs rôles 15
3.3 Établissement de session — protocole hybride 17
3.3.1 Étape 1 — Récupération du bundle 17
3.3.2 Étape 2 — Vérification des signatures 17
3.3.3 Étape 3 — Calcul du secret partagé hybride 17
3.3.4 Étape 4 — Récupération du secret côté Bob 17
3.3.5 Simplification par rapport au PQXDH complet 18
3.3.6 Diagramme de séquence 18
3.4 Chiffrement de chaque message 19
4. Mise en œuvre 20
4.1 Architecture logicielle 20
4.2 Génération et stockage des clés 22
4.3 Gestion des sessions 23
4.4 Interface et notifications push 23
5. Difficultés rencontrées et limitations 24

5.1 Difficultés techniques 24
5.2 Limitations de sécurité connues 24
6. Dimensions humaines, éthique, légale 26
6.1 Vie privée et impact humain 26
6.1.1 Le droit à la vie privée 26
6.1.2 Protection offerte par l'application 26
6.1.3 Le débat sécurité nationale vs vie privée 26
6.1.4 Accessibilité 26
6.2 Aspects légaux 27
6.2.1 Cadre canadien — LPRPDE et Loi 25 27
6.2.2 Licences open source 27
6.2.3 Interception légale 27
6.3 Considérations éthiques 27
6.3.1 Usage dual du chiffrement 27
6.3.2 Responsabilité du développeur 28
6.3.3 Choix éthiques de conception 28
7. Conclusion 29
7.1 Résumé des réalisations 29
7.2 Limites et recommandations 29
Annexe 1 — Workflow complet avec exemples numériques 30
Étape 0 — Bob s'inscrit (KeyManager.generateAndStore) 30
Étape 1 — Alice récupère les clés de Bob (GET /keys/bob) 31
Étape 2 — Alice calcule le secret partagé 32
Étape 3 — Alice chiffre le message (AppState.transmit) 33

Étape 4 — Bob déchiffre (SessionManager.decrypt) 34
Messages suivants 34
Annexe 2 — Visuel du serveur 35
Annexe 3 — Visuels de l’application 36
Références 38

1. Introduction

1.1 Contexte

Les applications de messagerie instantanée sont devenues un outil de communication quotidien pour des milliards d'utilisateurs. Pour protéger la confidentialité de ces échanges, le chiffrement de bout en bout (E2EE) s'est imposé comme standard, avec des protocoles comme celui de Signal qui font référence dans l'industrie.

Cependant, l'émergence des ordinateurs quantiques représente une menace fondamentale pour les systèmes cryptographiques actuels. Les algorithmes classiques tels que RSA et les courbes elliptiques (X25519, Ed25519) tirent leur sécurité de problèmes mathématiques considérés comme difficiles : la factorisation de grands nombres et le logarithme discret. L'algorithme de Shor, exécuté sur un ordinateur quantique suffisamment puissant, peut résoudre ces problèmes en temps polynomial, rendant ces protections obsolètes.

1.2 La menace "Harvest Now, Decrypt Later"

Cette menace n'est pas que théorique. Des acteurs étatiques et malveillants pratiquent déjà des attaques dites "harvest now, decrypt later" : ils interceptent et stockent massivement des communications chiffrées aujourd'hui, pariant sur la capacité de les déchiffrer dans 10 à 20 ans lorsque les ordinateurs quantiques seront opérationnels. Pour les données à longue durée de vie — secrets d'État, informations médicales, propriété intellectuelle — cette menace est déjà concrète.

Il existe historiquement un décalage entre les capacités cryptographiques publiques et celles des agences de renseignement. La NSA, par exemple, avait développé des techniques de cryptanalyse avancées bien avant leur publication académique [1]. Il est donc plausible que certaines entités disposent déjà de capacités quantiques partielles sans les divulguer publiquement. Cette incertitude renforce l'urgence d'adopter dès maintenant des algorithmes résistants aux attaques quantiques.

1.3 Standardisation post-quantique

En réponse à ce défi, le National Institute of Standards and Technology (NIST) a lancé en 2016 un processus de standardisation d'algorithmes cryptographiques résistants aux attaques quantiques [2]. En août 2024, les premiers standards ont été publiés :

- **FIPS 203 (ML-KEM)** [3] : Mécanisme d'encapsulation de clés basé sur les réseaux euclidiens (anciennement CRYSTALS-Kyber)
- **FIPS 204 (ML-DSA)** [4] : Algorithme de signature numérique basé sur les réseaux euclidiens (anciennement CRYSTALS-Dilithium)

- **FIPS 205 (SLH-DSA)** [5] : Algorithme de signature basé sur les fonctions de hachage (anciennement SPHINCS+)

1.4 Problématique

Bien que la recherche académique sur la cryptographie post-quantique soit abondante, il existe peu d'implémentations documentées sur plateformes mobiles. Les développeurs souhaitant migrer leurs applications vers la cryptographie post-quantique manquent de guides pratiques, particulièrement sur iOS.

Ce projet vise à répondre à la question suivante :

> **Comment implémenter une messagerie chiffrée de bout en bout utilisant des algorithmes post-quantiques sur iOS, et quels sont les compromis et défis rencontrés par rapport aux solutions classiques ?**

1.5 Objectifs

1. Implémenter une application iOS de messagerie utilisant une approche **hybride** combinant cryptographie classique (X25519, Ed25519) et post-quantique (ML-KEM, ML-DSA)
2. Concevoir une architecture de chiffrement de bout en bout adaptée aux contraintes mobiles
3. Permettre la création de compte par pseudonyme (sans numéro de téléphone)
4. Développer un serveur zero-knowledge pour le transit des messages
5. Documenter les difficultés d'intégration des librairies PQC sur iOS

2. État de l'art

2.1 La menace quantique sur la cryptographie actuelle

2.1.1 L'algorithme de Shor

En 1994, le mathématicien Peter Shor a démontré qu'un ordinateur quantique pouvait factoriser des nombres entiers et résoudre le problème du logarithme discret en temps polynomial [6]. Ces deux problèmes constituent le fondement de la sécurité des algorithmes asymétriques actuels :

- **RSA** repose sur la difficulté de factoriser le produit de deux grands nombres premiers
- **ECDH/ECDSA et X25519/Ed25519** reposent sur le problème du logarithme discret sur courbe elliptique

Un ordinateur quantique suffisamment puissant pourrait donc casser tous ces algorithmes. Les estimations actuelles parlent de quelques milliers de qubits logiques stables pour menacer RSA-2048.

2.1.2 L'algorithme de Grover

L'algorithme de Grover ne casse pas les algorithmes symétriques, mais réduit leur sécurité effective de moitié. AES-128 deviendrait l'équivalent de 64 bits de sécurité — insuffisant. AES-256, réduit à 128 bits effectifs, reste acceptable. Il suffit donc de doubler les longueurs de clés symétriques, pas de les remplacer entièrement.

2.1.3 Ce qui doit être remplacé

Tableau 1. Algorithmes et leur resistance post quantique

Categorie	Algorithme	Mathematiques	Menace
Non resistant	RSA, DH, DSA	Factorisation, Log discret	Shor
Non resistant	ECDH, ECDSA, X25519, Ed25519	Log discret, courbe elliptique	Shor
Resistant	AES-256, SHA-256	—	Gover
Resistant	ML-KEM, ML-DSA, SLH-DSA	Reseaux Euclidiens, hash	—

2.1.4 Déjà cassés classiquement

Certains algorithmes ne nécessitent même pas d'ordinateur quantique pour être compromis :

- **MD5** : collisions calculables en secondes
- **SHA-1** : collision démontrée par Google en 2017
- **DES** : brute-force faisable en heures
- **RC4** : attaques pratiques connues et documentées
- **3DES** : déprécié, vulnérable à l'attaque Sweet32
- **RSA-1024 et moins** : factorisable avec des ressources raisonnables

La règle générale : tout algorithme d'échange de clé ou de signature reposant sur la factorisation ou le logarithme discret est à remplacer. Les algorithmes symétriques (AES, SHA) sont à conserver en utilisant des clés d'au moins 256 bits.

2.2 Standards NIST post-quantiques

2.2.1 ML-KEM (FIPS 203)

ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) [3], anciennement CRYSTALS-Kyber, est un mécanisme d'encapsulation de clés. Contrairement à un échange Diffie-Hellman classique, un KEM fonctionne ainsi :

1. Bob génère une paire de clés (publique/privée)
2. Alice utilise la clé publique de Bob pour **encapsuler** un secret partagé, produisant un ciphertext
3. Bob utilise sa clé privée pour **décapsuler** le ciphertext et récupérer le secret partagé

La sécurité de ML-KEM repose sur le problème Module-LWE (Learning With Errors), considéré comme résistant aux attaques quantiques [7]. Trois niveaux de sécurité sont définis : ML-KEM-512 (équivalent AES-128), ML-KEM-768 (équivalent AES-192) et ML-KEM-1024 (équivalent AES-256). Ce projet utilise ML-KEM-768, offrant un bon compromis entre sécurité et taille des clés pour un prototype mobile.

2.2.2 ML-DSA (FIPS 204)

ML-DSA (Module-Lattice-Based Digital Signature Algorithm) [4], anciennement CRYSTALS-Dilithium, est un algorithme de signature numérique. Il permet de signer des données avec une clé privée et de vérifier l'authenticité avec la clé publique correspondante. Comme ML-KEM, sa

sécurité repose sur des problèmes de réseaux euclidiens [7]. Ce projet utilise ML-DSA-65, le niveau intermédiaire, équivalent à environ 192 bits de sécurité classique.

2.3 Protocoles de messagerie sécurisée existants

2.3.1 Le protocole Signal

Le protocole Signal, développé par Open Whisper Systems, est considéré comme la référence en matière de messagerie sécurisée [8]. Il combine deux mécanismes principaux.

X3DH (Extended Triple Diffie-Hellman) [9] gère l'établissement initial d'une session sécurisée entre deux utilisateurs, même si l'un d'eux est hors ligne. Il utilise plusieurs types de clés : une Identity Key permanente, une Signed PreKey semi-permanente signée par l'IK, et des One-Time PreKeys éphémères à usage unique.

Double Ratchet [10] assure le renouvellement continu des clés de chiffrement après l'établissement de la session. Chaque message utilise une clé différente, garantissant deux propriétés : la **forward secrecy** (la compromission d'une clé ne permet pas de déchiffrer les messages passés) et la **Post-Compromise Security** (après une compromission, la sécurité est restaurée lors des échanges suivants).

2.3.2 PQXDH

Signal a publié PQXDH (Post-Quantum Extended Diffie-Hellman) [11] comme évolution post-quantique de X3DH. L'approche est **hybride** : elle combine les échanges classiques sur courbes elliptiques avec une encapsulation ML-KEM. Le secret final est dérivé des deux composantes. Les clés de Bob incluent désormais des clés PQ (PQSPK, PQOPK utilisant ML-KEM) en plus des clés classiques.

2.3.3 Apple PQ3

En février 2024, Apple a déployé PQ3 pour iMessage [12], qualifié de "niveau 3" de sécurité post-quantique. Comme PQXDH, PQ3 utilise une approche hybride combinant courbes elliptiques et ML-KEM, avec un renouvellement périodique des clés PQ.

2.3.4 Fonctionnement hybride — illustration

Le principe de l'approche hybride est que le secret final dépend de **deux composantes indépendantes**. Pour compromettre la session, un attaquant doit casser les deux simultanément.

Supposons qu'Alice calcule : - un secret classique **S_c = 2** via X25519 (Diffie-Hellman, voir section 3.1.2) - un secret post-quantique **S_pq = 7** via ML-KEM (encapsulation, voir section 3.1.5)

Le secret final est `masterSecret = HKDF(2 || 7) = 42`.

Scénario 1 — Un ordinateur quantique casse X25519 et retrouve S_c = 2. Mais sans connaître S_pq = 7 (protégé par ML-KEM, résistant aux quantiques), l'attaquant ne peut pas reconstruire le masterSecret.

Scénario 2 — Une faille est découverte dans ML-KEM. L'attaquant retrouve S_pq = 7. Mais sans S_c = 2 (protégé par X25519, toujours sûr contre les attaques classiques), le masterSecret reste inaccessible.

Les deux algorithmes doivent être compromis **en même temps** pour que la session soit exposée. C'est la logique de défense en profondeur adoptée par Signal [11] et Apple [12].

2.3.5 Comparatif des applications existantes

Tableau 2. Systèmes d'encryption d'applications connues

Application	E2EE	Post-Quantique	Approche
Signal	Oui	Oui, PQXDH	Hybride
iMessage	Oui	Oui, PQ3	Hybride
WhatsApp	Oui	Non	Classique
Telegram	Optionnel	Non	Classique

2.3.6 Lacunes identifiées

Malgré les avancées de Signal et Apple, plusieurs lacunes persistent pour les développeurs tiers :

1. **Documentation limitée** : Les implémentations de Signal et Apple sont peu documentées pour les développeurs souhaitant reproduire des solutions similaires.
2. **Peu d'exemples sur mobile** : La majorité des ressources sur la cryptographie post-quantique concernent les environnements serveur ou desktop. Les contraintes spécifiques aux appareils mobiles (gestion du Keychain, taille des clés, stockage) sont peu abordées.
3. **Dépendance au numéro de téléphone** : Signal et WhatsApp requièrent un numéro de téléphone pour l'inscription, ce qui pose des problèmes de vie privée et d'accessibilité.

Ce projet vise à combler ces lacunes en proposant une implémentation hybride documentée sur iOS, avec inscription par pseudonyme.

2.4 Librairies iOS disponibles

Pour intégrer du PQC sur iOS native en 2025, les options disponibles sont :

Tableau 3. Composants de l'encryption

Composant	Librairie	Source
ML-KEM	SwiftKyber	Leif Ibsen, Open source MIT
ML-DSA	SwiftDilithium	Leif Ibsen, Open source MIT
X25519, Ed25519, AES, HKDF	Apple CryptoKit	Apple iOS

Limite identifiée : SwiftKyber et SwiftDilithium sont maintenues par un développeur indépendant et n'ont pas fait l'objet d'un audit de sécurité formel. Pour une application de production dans un environnement sensible, deux alternatives seraient recommandées :

- **liboqs** (Open Quantum Safe) [16] : librairie C plus largement auditée, utilisée comme référence dans le processus NIST
- **Apple CryptoKit ML-KEM** [17] : intégration native annoncée pour iOS 26 (automne 2026)

Ces librairies sont acceptables pour un prototype académique, mais cette limitation doit être signalée dans toute évaluation de déploiement.

3. Architecture et protocole

3.1 Les algorithmes cryptographiques utilisés

Le projet utilise six algorithmes, répartis en deux catégories.

3.1.1 Ed25519 — signatures classiques

Ed25519 est un algorithme de signature numérique [18]. Son rôle est d'authentifier l'identité : prouver que c'est bien Bob qui a créé une clé donnée. Il repose sur le problème du logarithme discret sur une courbe elliptique de type Edwards (d'où le "Ed"). La courbe est définie sur le nombre premier $2^{255} - 19$, d'où le suffixe "25519". RSA remplit le même rôle, mais Ed25519 offre des clés bien plus compactes (32 octets vs 256 octets pour RSA-2048) pour un niveau de sécurité équivalent. Les deux sont vulnérables aux ordinateurs quantiques.

Notre code appelle simplement : `Curve25519.Signing.PrivateKey()` — les paramètres de la courbe sont dans CryptoKit [15]. Pour illustrer le principe avec des petits nombres (analogue RSA-like, n=33) :

```
Bob choisit IK_priv = 7   → Keychain (jamais partagé)
Bob calcule IK_pub  = 5^7 mod 23 = 3   → serveur

Signer SPK_pub=19 :   sig = 19^7 mod 33 = 13   → serveur
Vérifier (Alice) :    13^3 mod 33 = 19 = SPK_pub ✓
Si SPK_pub modifié (19→20) : 13^3 mod 33 ≠ 20 → refus
```

3.1.2 X25519 — échange de clés classique

X25519 est un algorithme d'échange de clés Diffie-Hellman [18]. Son rôle est de permettre à deux parties de se mettre d'accord sur un secret commun sans jamais le transmettre sur le réseau. Il utilise la même courbe de base (Curve25519), mais dans une représentation adaptée à l'échange de clés — seule la coordonnée X du point sur la courbe est utilisée (d'où le "X"). Vulnérable aux ordinateurs quantiques.

Notre code appelle simplement : `Curve25519.KeyAgreement.PrivateKey()` — les paramètres (p=grand premier, g=générateur) sont dans CryptoKit [15]. Pour illustrer avec des petits nombres (p=23, g=5) :

```
Alice : eph_priv=6  → eph_pub = 5^6 mod 23 = 8    → envoyé à Bob
Bob   : SPK_priv=15 → SPK_pub = 5^15 mod 23 = 19  → envoyé à Alice

Alice calcule : 19^6  mod 23 = 2
```

```
Bob calcule   :  8^15 mod 23 = 2  ✓
→ secret commun = 2, sans jamais être transmis
```

3.1.3 AES-256-GCM — chiffrement symétrique

AES-256-GCM est l'algorithme qui chiffre effectivement chaque message [19]. AES (Advanced Encryption Standard) en mode GCM (Galois/Counter Mode) combine le chiffrement par blocs en mode compteur et un tag d'authentification de 16 octets (GMAC), garantissant à la fois la confidentialité et l'intégrité. Résistant aux ordinateurs quantiques avec des clés de 256 bits.

```
msgKey = 17,  nonce = 4F3A...  (12 octets aléatoires, usage unique)

AES-GCM.Encrypt("Salut Bob !", key=17, nonce=4F3A...) = [B7 2C 91 ...]
+ tag 16 octets
AES-GCM.Decrypt([B7 2C 91 ...], key=17, nonce=4F3A...) = "Salut Bob !"
✓

Avec mauvaise clé (key=18)    → déchiffrement produit du bruit
Si ciphertext modifié en transit → tag invalide → message rejeté
```

3.1.4 HKDF-SHA256 — dérivation de clés

HKDF (HMAC-based Key Derivation Function) ne chiffre rien directement. Il transforme un matériel de clé brut en une ou plusieurs clés de longueur et format adaptés, en utilisant un contexte ("info") pour que deux usages du même matériel produisent des clés distinctes. Résistant aux ordinateurs quantiques.

```
masterSecret = 42  (Keychain — ne change pas pendant toute la session)

msgKey_0 = HKDF(42, "msg"||0) = 17   ← chiffre le message 0
msgKey_1 = HKDF(42, "msg"||1) = 53   ← chiffre le message 1
msgKey_2 = HKDF(42, "msg"||2) = 31   ← chiffre le message 2

→ même masterSecret, clé différente à chaque message, jamais stockées
```

3.1.5 ML-KEM-768 — encapsulation post-quantique

ML-KEM est un mécanisme d'encapsulation de clés post-quantique [3] [7]. Contrairement à Diffie-Hellman, il ne fait pas d'échange : Alice génère un secret aléatoire, le "verrouille" avec la clé publique de Bob (encapsulation), et seul Bob peut le "déverrouiller" avec sa clé privée (décapsulation). Sa sécurité repose sur les réseaux euclidiens (lattices), un problème mathématique pour lequel aucun algorithme quantique efficace n'est connu. Le niveau 768 offre une sécurité équivalente à AES-192.

Notre code appelle : `Kyber.GenerateKeyPair(kind: .K768)` [13]. Pour illustrer (les vrais paramètres sont dans SwiftKyber) :

```
Bob génère : PQSPK_pub=🔒 → serveur   PQSPK_priv=🔑 → Keychain

Alice : (kemCT, ss) = Encaps(🔒)
  ss    = 7          ← secret aléatoire
  kemCT = [7🔒]      ← ciphertext envoyé à Bob avec le message

Bob : ss = Decaps([7🔒], 🔑) = 7  ✓
→ [7🔒] intercepté en transit est inexploitable sans 🔑
```

3.1.6 ML-DSA-65 — signatures post-quantiques

ML-DSA est l'algorithme de signature numérique post-quantique [4] [7], analogue post-quantique d'Ed25519. Même rôle — authentifier l'identité — mais reposant sur les réseaux euclidiens. Ses signatures sont environ 50 fois plus volumineuses qu'Ed25519 (3 293 octets vs 64 octets), ce qui est le principal coût de la résistance quantique pour les signatures.

Notre code appelle : `Dilithium.GenerateKeyPair(kind: .ML_DSA_65)` [14]. Pour illustrer :

```
Bob génère : PQIK_priv=🖋 → Keychain   PQIK_pub=🔍 → serveur

Signer PQSPK_pub=🔒 :   sig_pq = 🖋 signe 🔒 = ✍6   → serveur
Vérifier (Alice) :       🔍 vérifie ✍6 sur 🔒 ✓
Si 🔒 a été modifié en transit → ✍6 ne correspond plus → refus
```

3.2 Types de clés et leurs rôles

À l'inscription, chaque utilisateur génère sept types de clés.

Identity Key (IK) — Ed25519 Clé d'identité permanente. Sert uniquement à signer les autres clés. Prouve que c'est bien l'utilisateur qui a créé le SPK et le PQSPK. Ne participe jamais directement au chiffrement des messages. Durée de vie : compte entier.

PQ Identity Key (PQIK) — ML-DSA-65 Même rôle que l'IK, mais résistant aux ordinateurs quantiques. Les deux signent les PreKeys simultanément — double garantie d'authenticité. Si Ed25519 est cassé dans le futur, ML-DSA tient encore. Durée de vie : compte entier.

Signed PreKey (SPK) — X25519 Clé semi-permanente. C'est elle que l'envoyeur utilise pour le Diffie-Hellman lors de l'établissement du secret commun. "Signée" signifie que l'IK et le PQIK ont certifié que cette clé appartient bien à l'utilisateur, ce que tout destinataire peut vérifier. Durée de vie prévue par le protocole Signal [9] : environ une semaine (non implémenté dans ce prototype — voir section 5.2).

PQ Signed PreKey (PQSPK) — ML-KEM-768 Même rôle que le SPK, mais pour la partie post-quantique. L'envoyeur encapsule un secret avec cette clé publique. Seul le détenteur de la clé privée correspondante peut le décapsuler.

One-Time PreKeys × 5 (OPK) — X25519 Clés jetables — chacune n'est utilisée qu'une seule fois puis détruite. Leur but est de fournir de la **forward secrecy classique** : si une session est compromise, les autres sessions (qui ont utilisé des OPK différentes) restent protégées. Après épuisement des 5 OPKs, le handshake tombe en fallback sur le SPK permanent.

PQ One-Time PreKeys × 5 (PQOPK) — ML-KEM-768 Même idée que les OPK mais pour ML-KEM. Dans ce projet, par souci de simplification de l'implémentation, elles sont générées et uploadées sur le serveur mais non intégrées dans la dérivation du secret — le PQSPK permanent est utilisé à la place. En production, elles devraient être intégrées au même titre que les OPK classiques pour assurer la forward secrecy post-quantique par session.

Tableau 4. Synthèse des stockages de clé utilisés

Quoi	Où	Sécurité
masterSecret	KeyChain iOS	Chiffré
Clé Publique	KeyChain iOS + Sever	Non sensible
authToken	UserDefaults	Non chiffré
Message	UserDefaults	Non chiffré
Clé éphémère privée	Mémoire	Éphémère
Message Key	Mémoire	Usage Unique

3.3 Établissement de session — protocole hybride

Lorsqu'Alice souhaite envoyer un premier message à Bob, le protocole se déroule en quatre étapes [11].

3.3.1 Étape 1 — Récupération du bundle

Alice appelle `GET /keys/bob`. Le serveur retourne le bundle de clés publiques de Bob et **consomme une OPK** (elle est supprimée du serveur, usage unique).

3.3.2 Étape 2 — Vérification des signatures

Avant tout calcul, Alice vérifie que les clés reçues sont bien authentiques : - Ed25519 : vérification que SPK_bob et PQSPK_bob ont été signés par IK_bob - ML-DSA : vérification des mêmes signatures par PQIK_bob

Si une vérification échoue, la session est refusée. Cette étape protège contre un serveur compromis qui injecterait de fausses clés.

3.3.3 Étape 3 — Calcul du secret partagé hybride

Alice génère une clé éphémère X25519 (usage unique, jamais stockée). Elle calcule :

```
dh          = X25519(eph_priv, SPK_bob_pub)
dh_opk      = X25519(eph_priv, OPK_bob_pub)   (si OPK disponible)
(kemCT, ss) = ML-KEM.Encaps(PQSPK_bob_pub)
ikm          = dh || dh_opk || ss
masterSecret = HKDF-SHA256(ikm, info="hybrid_session_v1")
```

Le résultat est un masterSecret de 32 octets. Alice envoie à Bob le message chiffré accompagné de sa clé publique éphémère et du ciphertext ML-KEM (`kemCT`).

3.3.4 Étape 4 — Récupération du secret côté Bob

Bob charge ses clés privées depuis le Keychain (SPK_priv, PQSPK_priv, OPK_priv si utilisée). Il effectue les mêmes opérations en miroir :

```
dh          = X25519(SPK_priv, eph_pub)
dh_opk      = X25519(OPK_priv, eph_pub)       (OPK_priv détruite
après)
ss          = ML-KEM.Decaps(kemCT, PQSPK_priv)
masterSecret = HKDF-SHA256(dh || dh_opk || ss, "hybrid_session_v1")
```

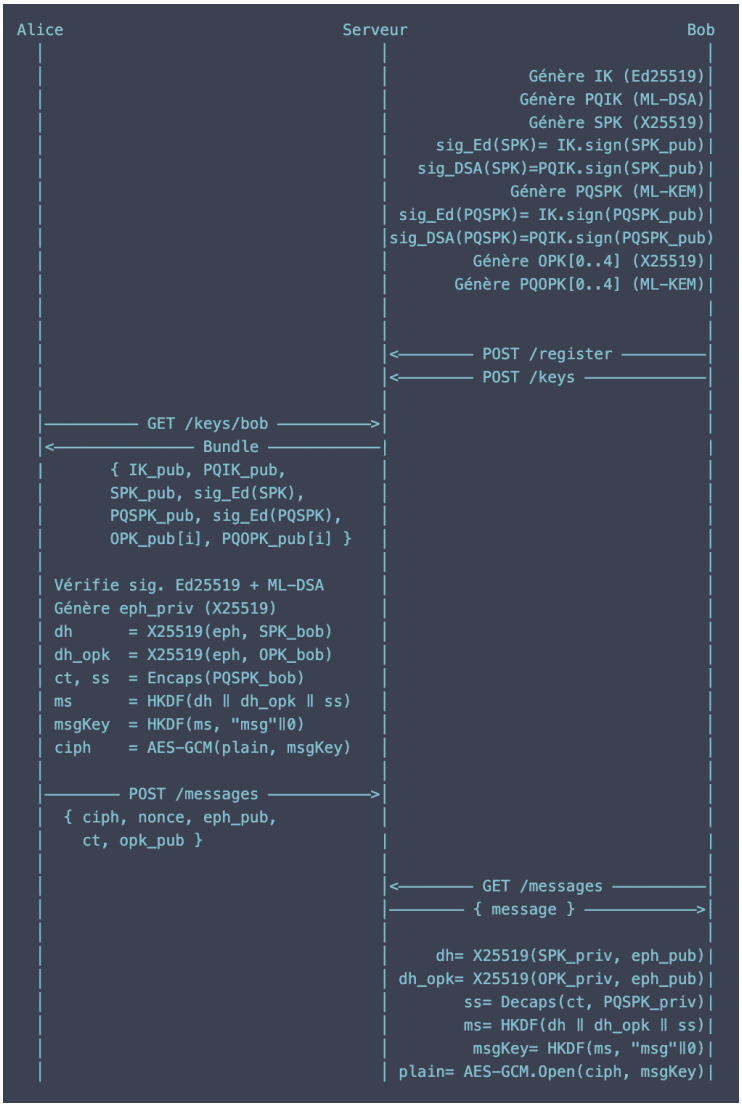
Bob obtient exactement le même masterSecret qu'Alice, sans qu'il ait jamais été transmis sur le réseau. Les deux parties sauvegardent ce secret dans leur Keychain respectif.

3.3.5 Simplification par rapport au PQXDH complet

Dans le protocole PQXDH de Signal [11], la clé d'identité d'Alice (IK_alice) participe également aux échanges DH, liant cryptographiquement l'identité de l'expéditeur au secret établi. Cette liaison a été retirée dans ce prototype pour simplifier l'implémentation. Conséquence : aucune signature cryptographique ne prouve que le message vient bien de l'IK d'Alice — limitation documentée en section 5.2.

3.3.6 Diagramme de séquence

Figure 1. Diagramme de séquences d'établissement de communication



3.4 Chiffrement de chaque message

Le masterSecret ne chiffre **jamais directement** les messages. Pour chaque message, une clé unique est dérivée :

```
msgKey_N = HKDF-SHA256(masterSecret, info = "msg" || N)
```

Où N est un compteur incrémenté à chaque message envoyé (compteurs distincts pour l'envoi et la réception).

Le message est ensuite chiffré avec AES-256-GCM [19] :

```
nonce      = 12 octets aléatoires (usage unique)
ciphertext = AES-256-GCM.Encrypt(plaintext, key=msgKey_N, nonce)
           → ciphertext + tag d'authentification 16 octets
```

Le tag GCM garantit que si le ciphertext est modifié en transit, le déchiffrement échouera et le message sera rejeté. La msgKey n'est jamais stockée — elle est recalculée à chaque accès.

Le paquet envoyé au serveur contient : le ciphertext, le nonce, et (uniquement pour le premier message) la clé publique éphémère et le ciphertext ML-KEM.

4. Mise en œuvre

4.1 Architecture logicielle

Figure 2. Arbre de fichier de l'application

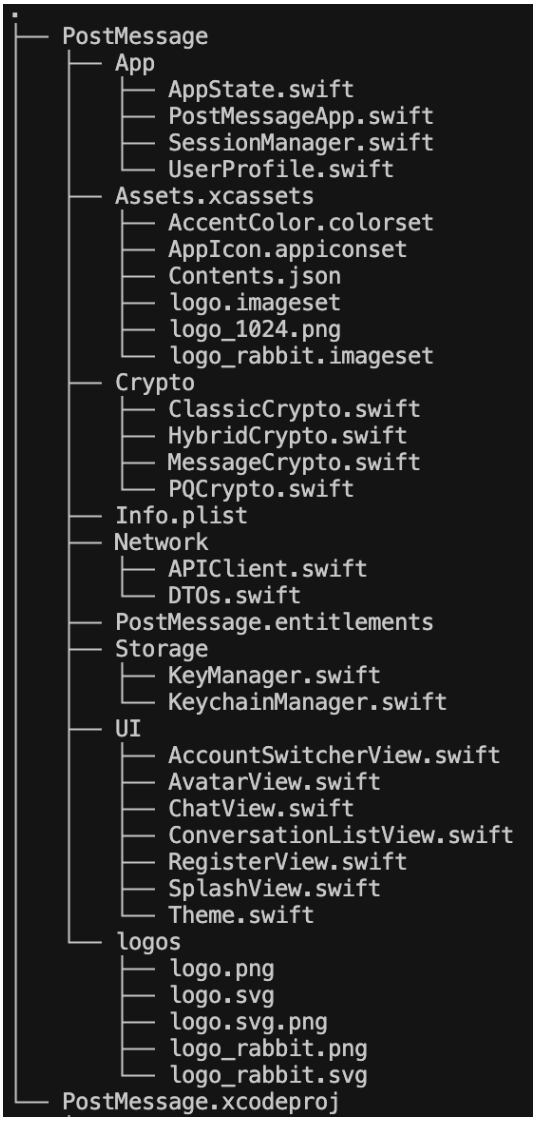
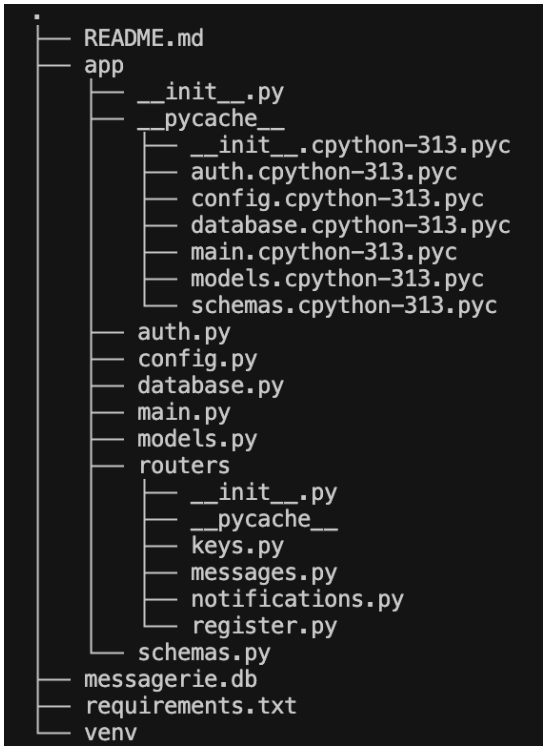


Figure 3. Arbre de fichier du serveur



L'application est organisée en quatre couches.

Couche UI (SwiftUI) `RegisterView`, `ConversationListView`, `ChatView`, `AccountSwitcherView`. Interface déclarative, sans logique cryptographique.

Couche application (AppState, SessionManager) `AppState` est le point central de l'état : gestion des profils, conversations, polling des messages (toutes les 3 secondes en tâche de fond persistante), compteurs de non-lus. `SessionManager` orchestre les échanges cryptographiques : établissement des sessions (handshake), déchiffrement des messages entrants.

Couche crypto - `ClassicCrypto.swift` : signatures Ed25519 et échange X25519 - `PQCrypto.swift` : encapsulation/décapsulation ML-KEM et signatures ML-DSA - `HybridCrypto.swift` : combinaison DH + KEM + HKDF pour le handshake - `MessageCrypto.swift` : chiffrement/déchiffrement AES-256-GCM

Couche stockage - `KeyManager.swift` : génération et accès aux clés - `KeychainManager.swift` : interface bas niveau vers l'API Keychain iOS

Backend Serveur Python FastAPI + SQLite. Principe zero-knowledge : le serveur ne stocke que des blobs chiffrés et ne peut pas accéder au contenu des messages.

Endpoints principaux :

Tableau 5. Endpoints de l'API du server

Endpoint	Méthode	Description
/register	POST	Créer compte
/keys	POST	Publier ses clés
/keys/{user}	GET	Demander bundle d'un user
/messages	POST	Envoyer message
/messages	GET	Recevoir messages en attente
/apns-token	POST	Enregistrer token de notification push

4.2 Génération et stockage des clés

À l'inscription, `KeyManager.generateAndStore()` génère l'ensemble des clés et les stocke dans le Keychain iOS. Chaque entrée Keychain est identifiée par le service `"com.postmessage.keys"` et un compte préfixé par l'identifiant utilisateur.

L'accès au Keychain passe par trois méthodes dans `KeychainManager.swift` utilisant l'API iOS `Security` : - `save()` : `SecItemDelete` + `SecItemAdd` (écrase si existe déjà) - `load()` : `SecItemCopyMatching` - `delete()` : `SecItemDelete`

Figure 4. Récapitulatif des clés, leur algorithme et leur lieu de stockage

GÉNÉRATION	ALGO	STOCKAGE
Identity Key (IK)		
├─ privée	Ed25519	Keychain(KC)
└─ publique	Ed25519	KC + serveur
PQ Identity Key (PQIK)		
├─ privée	ML-DSA-65	Keychain
└─ publique	ML-DSA-65	KC + serveur
Signed PreKey (SPK)		
├─ privée	X25519	Keychain
├─ publique	X25519	KC + serveur
├─ signature Ed25519 (par IK)	Ed25519	KC + serveur
└─ signature ML-DSA (par PQIK)	ML-DSA-65	KC + serveur
PQ Signed PreKey (PQSPK)		
├─ privée	ML-KEM-768	Keychain
├─ publique	ML-KEM-768	KC + serveur
├─ signature Ed25519 (par IK)	Ed25519	KC + serveur
└─ signature ML-DSA (par PQIK)	ML-DSA-65	KC + serveur
One-time PreKeys × 5 (OPK)		
├─ privées [0..4]	X25519	Keychain
└─ publiques [0..4]	X25519	KC + serveur
PQ One-time PreKeys × 5 (PQOPK)		
├─ privées [0..4]	ML-KEM-768	Keychain
└─ publiques [0..4]	ML-KEM-768	KC + serveur

4.3 Gestion des sessions

`SessionManager` gère le cycle de vie des sessions cryptographiques. Pour l'envoi, `getOrCreateSendSession()` récupère le bundle du destinataire, vérifie les signatures, et appelle `HybridCrypto.establishSession()` si aucune session existante n'est trouvée. Le masterSecret est sauvegardé dans le Keychain sous `sessions.<uid>` en JSON base64.

Pour la réception, `decrypt()` détecte si le message entrant contient une clé éphémère et un ciphertext ML-KEM (handshake initial) ou non (message suivant). Dans le premier cas, il reconstruit le masterSecret via `HybridCrypto.recoverSession()`. Dans les deux cas, il dérive la clé du message via `HybridCrypto.messageKey()` et déchiffre avec `MessageCrypto.decrypt()`.

Les sessions utilisent des compteurs distincts pour l'envoi (`sendCounter`) et la réception (`recvCounter`), afin d'éviter les conflits en conversation bidirectionnelle.

4.4 Interface et notifications push

L'interface utilise `NavigationStack` avec navigation programmatique. La liste des conversations trie automatiquement par timestamp du dernier message. Les pastilles de non-lus sont gérées par `AppState.unreadCounts`.

Le polling des messages est centralisé dans `AppState` comme tâche de fond persistante (intervalle de 3 secondes), indépendante des vues affichées. Les notifications push APNs sont intégrées pour alerter l'utilisateur quand l'app est en arrière-plan.

5. Difficultés rencontrées et limitations

5.1 Difficultés techniques

Race condition sur le polling Le polling toutes les 5 secondes depuis une vue SwiftUI causait deux appels `fetchMessages` simultanés. Les mêmes messages étaient déchiffrés deux fois, désynchronisant les compteurs de session. Résolution : flag `isFetching` dans `AppState` + tracking des IDs de messages déjà traités.

Compteurs de session partagés Un seul compteur par session causait des erreurs de déchiffrement en conversation bidirectionnelle : chaque côté incrémentait le même compteur, décalant les clés dérivées. Résolution : compteurs `sendCounter` et `recvCounter` distincts dans la structure `Session`. Cette correction est distincte du Double Ratchet [10] — le Double Ratchet régénèrerait les clés DH à chaque échange ; ici le masterSecret reste fixe, seule la gestion du compteur était incorrecte.

Sessions perdues au redémarrage `SessionManager` était un acteur en mémoire uniquement — toutes les sessions étaient perdues à la fermeture de l'app. Les messages suivants, sans données de handshake, échouaient systématiquement. Résolution : persistance des sessions (masterSecret + compteurs) dans le Keychain en JSON base64.

5.2 Limitations de sécurité connues

Ce prototype est fonctionnel mais présente plusieurs limitations de sécurité importantes à considérer avant tout déploiement.

Transport HTTP sans TLS Le serveur tourne en HTTP local. Les tokens d'authentification (Bearer) transitent en clair — un attaquant sur le même réseau peut les intercepter et usurper un compte. Le contenu des messages reste protégé par l'E2EE, mais l'accès au compte est exposé. En production : HTTPS avec TLS 1.3 obligatoire.

Historique des conversations en clair sur disque Les messages déchiffrés sont sérialisés en JSON dans `UserDefaults` — un fichier plist non chiffré dans le sandbox iOS, accessible par analyse forensique d'un appareil. Le masterSecret lui-même est dans le Keychain (chiffré). Correction recommandée : stocker les conversations chiffrées dans le Keychain ou utiliser SQLite avec SQLCipher.

Pas d'authentification de l'expéditeur Aucune signature cryptographique ne prouve que le message vient bien de l'IK du contact. Un serveur compromis peut injecter des messages arbitraires. Correction : signer chaque message avec l'IK de l'expéditeur et vérifier côté réception.

Pas de Double Ratchet Un seul masterSecret par conversation pour toute sa durée de vie. Si le Keychain est compromis, tous les messages passés et futurs de la session sont exposés. Le Double Ratchet [10] régénère des clés DH à chaque échange, limitant la fenêtre de compromission.

Pas de rotation du SPK Le Signed PreKey (X25519) est généré à l'inscription et jamais renouvelé. Le protocole X3DH de Signal [9] prévoit une rotation environ hebdomadaire. Il n'y a aucun timestamp, aucune tâche planifiée, aucune logique d'expiration dans le code. Correction : ajouter un timestamp à la création du SPK et déclencher une régénération si plus de 7 jours au prochain lancement.

PQOPK non intégrée dans le KEM Les ML-KEM one-time prekeys sont générées, uploadées et consommées par le serveur, mais jamais utilisées dans la dérivation du masterSecret. ML-KEM utilise toujours le PQSPK statique — aucune forward secrecy post-quantique par session individuelle.

Pas de renouvellement des OPKs Les 5 OPKs générées à l'inscription sont consommées sans être renouvelées. Quand la table est vide, le serveur tombe silencieusement en fallback sur le SPK permanent — sans signal au client. Correction : vérifier le stock d'OPKs restantes après chaque handshake et déclencher une régénération si moins de 2 OPKs disponibles.

Auth token permanent Le token généré à l'inscription ne expire jamais et ne peut pas être révoqué sans supprimer le compte. Correction : implémenter une expiration avec renouvellement.

Nettoyage incomplet à la suppression La suppression d'un compte en app ne déclenche pas d'appel `DELETE /account` côté serveur — le compte et les clés publiques restent actifs indéfiniment. La désinstallation de l'app supprime `UserDefaults` mais les items Keychain survivent par défaut sur iOS.

6. Dimensions humaines, éthique, légale

6.1 Vie privée et impact humain

6.1.1 Le droit à la vie privée

La vie privée est reconnue comme un droit fondamental par l'article 12 de la Déclaration universelle des droits de l'homme et l'article 8 de la Convention européenne des droits de l'homme. Dans un contexte de numérisation croissante des communications, le chiffrement de bout en bout est devenu un outil essentiel pour exercer ce droit.

Les révélations d'Edward Snowden en 2013 ont mis en lumière l'ampleur de la surveillance de masse pratiquée par certaines agences gouvernementales (NSA, GCHQ) [1], collectant des métadonnées et parfois le contenu des communications de millions de citoyens sans mandat individualisé.

6.1.2 Protection offerte par l'application

Le chiffrement E2EE offre une protection technique contre la surveillance de masse (le contenu des messages reste illisible même si intercepté), les fuites de données (un serveur compromis n'expose que des blobs chiffrés), et les accès non autorisés. Le choix du pseudonyme plutôt que du numéro de téléphone réduit la quantité de données personnelles identifiables collectées.

6.1.3 Le débat sécurité nationale vs vie privée

Les gouvernements et forces de l'ordre argumentent que le chiffrement fort entrave leurs capacités d'enquête. Certains proposent des "backdoors" ou des systèmes de surveillance côté client. La communauté cryptographique s'oppose généralement à ces propositions : une backdoor pour les autorités est aussi une backdoor pour des acteurs malveillants, et les criminels utiliseraient simplement d'autres outils.

Ce projet ne contient pas de backdoor et ne permet pas l'accès au contenu des messages par des tiers, y compris le développeur ou l'opérateur du serveur.

6.1.4 Accessibilité

Ce prototype cible iOS uniquement, ce qui exclut les utilisateurs Android et les plateformes sans smartphone. Une version Android et une interface web seraient nécessaires pour une accessibilité complète. Par ailleurs, la cryptographie post-quantique ajoute une charge lors de l'établissement des sessions (taille des clés et signatures plus importantes), ce qui pourrait affecter les appareils moins récents.

6.2 Aspects légaux

6.2.1 Cadre canadien — LPRPDE et Loi 25

La Loi sur la protection des renseignements personnels et les documents électroniques (LPRPDE) [20] est la loi fédérale canadienne régissant la collecte, l'utilisation et la communication de renseignements personnels dans le secteur privé. Elle impose notamment les principes de responsabilité, de consentement, de limitation de la collecte et de mesures de sécurité appropriées. Le chiffrement E2EE répond directement au principe de mesures de sécurité.

La Loi 25 du Québec [21] (Loi modernisant des dispositions législatives en matière de protection des renseignements personnels), entrée en vigueur progressivement depuis 2022, impose des obligations renforcées : évaluation des facteurs relatifs à la vie privée pour les projets à risque, notification obligatoire en cas d'incident de confidentialité, et droit à la portabilité des données.

L'approche adoptée (pseudonyme, E2EE, minimisation des données) est conforme aux exigences des deux cadres.

6.2.2 Licences open source

Les librairies utilisées sont sous licence MIT (SwiftKyber [13], SwiftDilithium [14]), permettant l'utilisation, la modification et la distribution libre, y compris dans des projets commerciaux, avec pour seule obligation de conserver la notice de copyright.

6.2.3 Interception légale

La Partie VI du Code criminel canadien (articles 183 à 196) [22] encadre l'interception des communications privées. L'interception requiert généralement une autorisation judiciaire. La loi canadienne n'impose actuellement pas aux développeurs d'intégrer des backdoors, de fournir des clés de déchiffrement (qu'ils ne possèdent pas dans un système E2EE), ou d'affaiblir leur chiffrement.

Des débats législatifs sont en cours dans plusieurs pays (Australie, Royaume-Uni, Union Européenne) concernant l'accès aux communications chiffrées. Le Canada n'a pas encore adopté de législation similaire.

6.3 Considérations éthiques

6.3.1 Usage dual du chiffrement

Le chiffrement fort protège autant les communications légitimes (journalistes, activistes, entreprises, citoyens ordinaires) que les communications illégales. La position adoptée dans ce projet est que les bénéfices pour la majorité écrasante des utilisateurs légitimes l'emportent sur

les risques d'abus, que les criminels disposent déjà d'alternatives, et que les backdoors créent plus de vulnérabilités qu'elles n'en résolvent.

6.3.2 Responsabilité du développeur

Plusieurs responsabilités éthiques s'appliquent : transparence (le code est open source), honnêteté (les limitations connues sont documentées et ne sont pas cachées), et minimisation des dommages (le pseudonyme évite l'anonymat total qui faciliterait le harcèlement non traçable).

6.3.3 Choix éthiques de conception

Tableau 6. Justification de choix éthiques

Choix	Justification
Pseudonyme choisi	Anonymat
Pas de numéro de téléphone	Accessibilité, multi compte
Server Zero-knowledge	Peu de risque de compromission
Limitations documentées	Transparence

7. Conclusion

7.1 Résumé des réalisations

Ce projet a produit une application iOS de messagerie chiffrée de bout en bout fonctionnelle, utilisant une approche hybride classique + post-quantique. Les réalisations concrètes incluent :

- Protocole de session hybride inspiré de PQXDH [11] : X25519 + ML-KEM-768 → HKDF → masterSecret
- Authentification double par signature Ed25519 + ML-DSA-65 des PreKeys
- Chiffrement de chaque message avec une clé dérivée unique (HKDF + AES-256-GCM)
- Infrastructure de clés complète dans le Keychain iOS (IK, PQIK, SPK, PQSPK, OPKs)
- Backend zero-knowledge avec SQLite
- Système de profils multiples, conversations persistées, notifications push APNs

7.2 Limites et recommandations

Les limitations principales du prototype sont documentées en section 5.2. Pour une application de production, les priorités seraient :

1. **HTTPS** : priorité absolue — les tokens d'authentification ne doivent pas transiter en clair
2. **Double Ratchet** [10] : régénération périodique des clés de session pour la forward secrecy et la Post-Compromise Security
3. **Rotation du SPK** : expiration hebdomadaire avec régénération automatique, comme spécifié dans X3DH [9]
4. **PQOPK dans le KEM** : intégrer les one-time prekeys ML-KEM dans la dérivation du masterSecret pour la forward secrecy post-quantique
5. **Stockage chiffré des messages** : Keychain ou SQLCipher plutôt que UserDefaults
6. **Librairies auditées** : migrer vers liboqs [16] ou Apple CryptoKit iOS 26 [17] pour un déploiement en environnement sensible

Annexe 1 — Workflow complet avec exemples numériques

Les algorithmes reposent sur des paramètres mathématiques fixes, définis dans des standards (NIST, RFC). Notre code ne les voit jamais — ils sont embarqués dans CryptoKit [15], SwiftKyber [13] et SwiftDilithium [14]. Pour illustrer le principe, on utilise des petits nombres.

Paramètres utilisés pour l'illustration

Pour l'échange DH (analogue X25519) : `p=23` (nombre premier), `g=5` (générateur). Notre code appelle `Curve25519.KeyAgreement.PrivateKey()` — ces paramètres sont dans CryptoKit.

```
Clé publique  = g ^ clé_privée mod p
Secret commun = pub_adverse  ^ clé_privée mod p
```

Pour les signatures (analogue Ed25519, simulé RSA-like) : `p=3, q=11, n=33`. Notre code appelle `Curve25519.Signing.PrivateKey()`.

```
Signer   : sig  = data ^ clé_privée mod 33
Vérifier : data = sig  ^ clé_publique mod 33
```

Pour ML-KEM et ML-DSA, la math des réseaux euclidiens est trop complexe pour de petits nombres. On utilise des symboles : 🔓 clé publique KEM, 🔑 clé privée KEM, 🖋 clé de signature DSA, 🔍 clé de vérification DSA. Notre code appelle `Kyber.GenerateKeyPair(kind: .K768)` et `Dilithium.GenerateKeyPair(kind: .ML_DSA_65)`.

Étape 0 — Bob s'inscrit (`KeyManager.generateAndStore`)

IK de Bob — Ed25519 (CryptoKit)

```
Bob choisit IK_priv = 7   → Keychain  (jamais partagé)
Bob calcule IK_pub  = 5^7 mod 23 = 3  → serveur
```

PQIK de Bob — ML-DSA (SwiftDilithium)

```
Bob génère PQIK_priv = 🖋️ → Keychain  (stylo pour signer)
           PQIK_pub  = 🔍 → serveur   (loupe pour vérifier)
```

SPK de Bob — X25519 (CryptoKit)

```
Bob choisit SPK_priv = 15
Bob calcule SPK_pub  = 5^15 mod 23 = 19   → serveur

Bob signe SPK_pub avec IK (Ed25519) :
  sig_classique = 19^7 mod 33 = 13   → serveur

Bob signe SPK_pub avec PQIK (ML-DSA) :
  sig_pq = 🖋️ signe 19 = ✍️9          → serveur
```

PQSPK de Bob — ML-KEM (SwiftKyber)

```
Bob génère PQSPK_pub  = 🔒 → serveur
           PQSPK_priv = 🔑 → Keychain

Bob signe 🔒 avec IK   : 🔒^7 mod 33 = 13   → serveur
Bob signe 🔒 avec PQIK : 🖋️ signe 🔒 = ✍️6  → serveur
```

OPKs de Bob — X25519 × 5 (CryptoKit)

```
OPK_priv[0] = 9,  OPK_pub[0] = 5^9 mod 23 = 11   → Keychain + serveur
OPK_priv[1] = 4,  OPK_pub[1] = 5^4 mod 23 = 4    → Keychain + serveur
OPK_priv[2..4] ...
```

PQOPKs de Bob — ML-KEM × 5 (SwiftKyber)

```
PQOPK[0..4] → Keychain + serveur
(générées et uploadées, non utilisées dans le KEM — limitation du
prototype)
```

Étape 1 — Alice récupère les clés de Bob (`GET /keys/bob`)

Le serveur retourne le bundle et **supprime OPK_pub[0] et PQOPK_pub[0]** (usage unique).

```
Reçu :
  IK_pub         = 3
  PQIK_pub       = 🔍
  SPK_pub        = 19,  sig_classique = 13,  sig_pq = ✍️9
  PQSPK_pub      = 🔒,  sig_classique = 13,  sig_pq = ✍️6
  OPK_pub[0]     = 11   ← supprimée du serveur
  PQOPK_pub[0]   = ?    ← supprimée du serveur, puis ignorée
```

Vérification des signatures — Ed25519 + ML-DSA

```
SPK_pub :
  Ed25519 : 13^3 mod 33 = 19 = SPK_pub ✓
  ML-DSA  : 🔍 vérifie ✍️9 sur 19 ✓

PQSPK_pub :
  Ed25519 : 13^3 mod 33 = 19 → vérifie 🔓 ✓
  ML-DSA  : 🔍 vérifie ✍️6 sur 🔓 ✓

→ Les clés viennent bien du vrai Bob. Si une vérification échoue → refus.
```

Étape 2 — Alice calcule le secret partagé

Clé éphémère — X25519

```
Alice choisit eph_priv = 6   → jamais stockée
Alice calcule eph_pub  = 5^6 mod 23 = 8   → sera envoyée à Bob
```

DH 1 — X25519 avec SPK

```
Alice : SPK_pub ^ eph_priv mod p = 19^6 mod 23 = 2
Bob   : eph_pub ^ SPK_priv mod p =  8^15 mod 23 = 2  ✓
dh = 2
```

DH 2 — X25519 avec OPK

```
Alice : OPK_pub[0] ^ eph_priv mod p = 11^6 mod 23 = 3
Bob   : eph_pub    ^ OPK_priv[0] mod p = 8^9 mod 23 = 3  ✓
dh_opk = 3
```

```
→ OPK_priv[0] = 9 détruite du Keychain de Bob après ce calcul (usage
unique)
```

Encapsulation — ML-KEM avec PQSPK

```
Alice choisit un secret aléatoire : ss = 7
Alice verrouille avec 🔒 : kemCiphertext = [7🔒]  → envoyé à Bob
Bob déverrouille [7🔒] avec 🔑 → retrouve 7
secret_pq = 7
```

Combinaison — HKDF

```
ikm          = dh || dh_opk || secret_pq
             = 2  || 3      || 7
masterSecret = HKDF(ikm, "hybrid_session_v1") = 42
→ sauvegardé dans le Keychain des deux côtés (sessions.<uid>)
```

Étape 3 — Alice chiffre le message (`AppState.transmit`)

Clé du message — HKDF

```
msgKey_0 = HKDF(masterSecret=42, "msg"||0) = 17   ← pour ce message
msgKey_1 = HKDF(masterSecret=42, "msg"||1) = 53   ← prochain message
→ jamais stockées, recalculées à chaque fois
```

Chiffrement — AES-256-GCM

```
plaintext = "Salut Bob !"
nonce     = 4F3A...  (12 octets aléatoires, usage unique)
ciphertext + tag = AES-GCM("Salut Bob !", key=17, nonce=4F3A...) = [B7
2C 91 ...] + tag
→ tag garantit que si le ciphertext est modifié en transit, le
déchiffrement échoue
```

Alice envoie au serveur :

```
{
  "eph_pub"        : 8,
  "kem_ciphertext" : "[7🔒]",
  "used_opk_pub"   : 11,
  "ciphertext"     : "B7 2C 91 ...",
```

```
  "nonce"          : "4F3A..."
}
```

Étape 4 — Bob déchiffre (`SessionManager.decrypt`)

```
Bob charge depuis Keychain :
  SPK_priv    = 15       ← Keychain "<uid>.spk.priv"
  PQSPK_priv = 🔑    ← Keychain "<uid>.pqspk.priv"
  OPK_priv[0] = 9        ← Keychain "<uid>.opk.priv.0" → DELETE après
usage

Refait les calculs :
  dh      = 8^15 mod 23 = 2
  dh_opk = 8^9  mod 23 = 3
  ss      = Decaps([7🔒], 🔑) = 7

  masterSecret = HKDF(2 || 3 || 7, "hybrid_session_v1") = 42  ✓

  msgKey_0 = HKDF(42, "msg"||0) = 17
  plaintext = AES-GCM.Open([B7 2C 91 ...], key=17, nonce=4F3A...) =
"Salut Bob !"
```

Bob retrouve “Salut Bob !” sans que le secret 42 n’ait jamais transité sur le réseau.

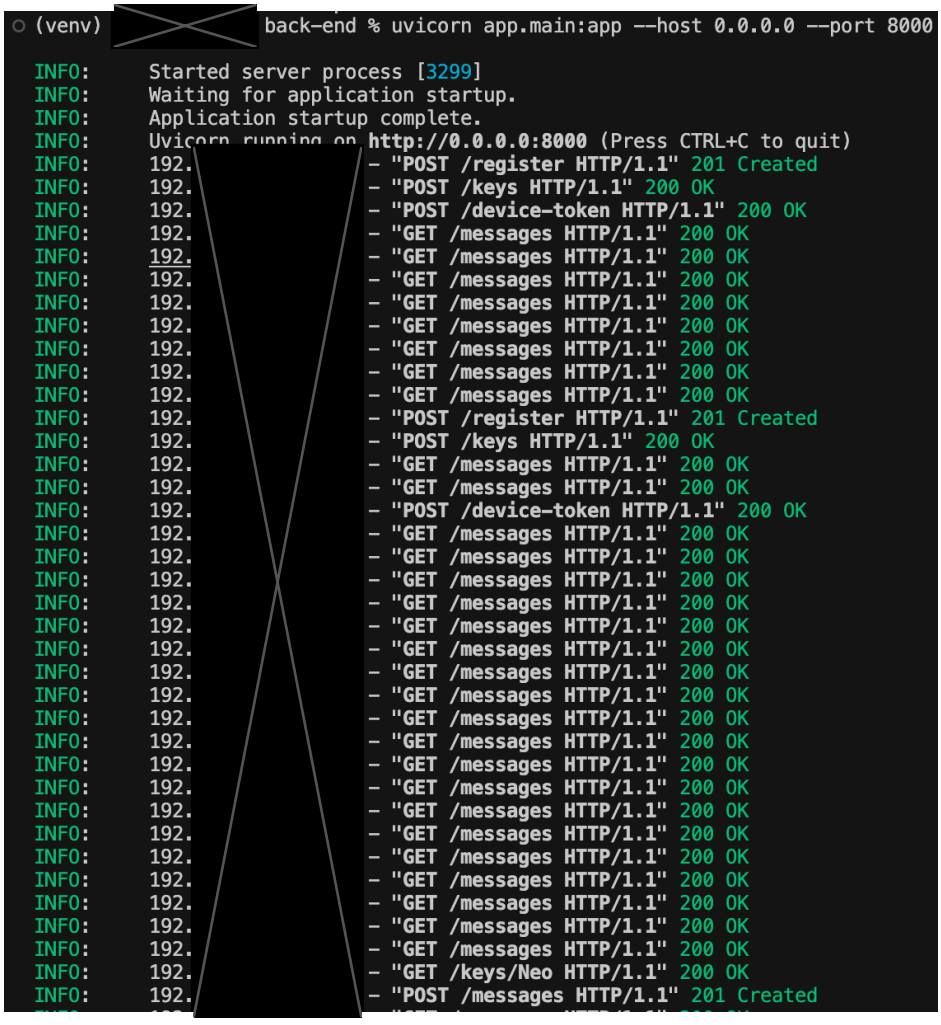
Messages suivants

Plus de handshake. `SessionManager` retrouve la session existante, utilise le compteur approprié (sendCounter côté Alice, recvCounter côté Bob) :

```
Message N :
  msgKey_N = HKDF(masterSecret=42, "msg"||N)
  → clé différente pour chaque message, jamais stockée
```

Annexe 2 — Visuel du serveur

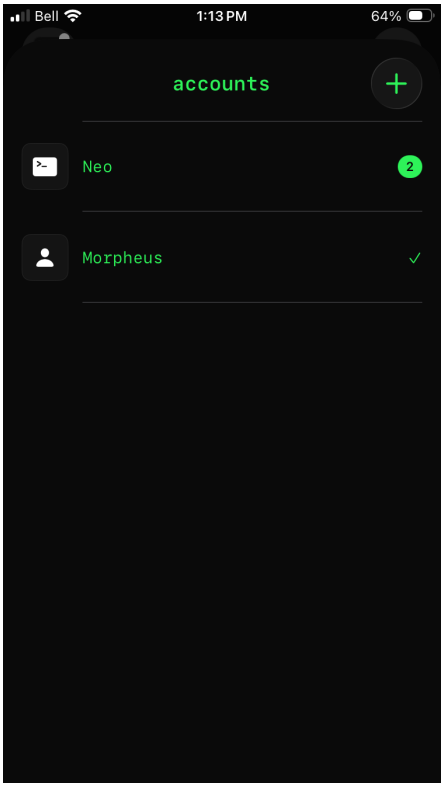
Terminal



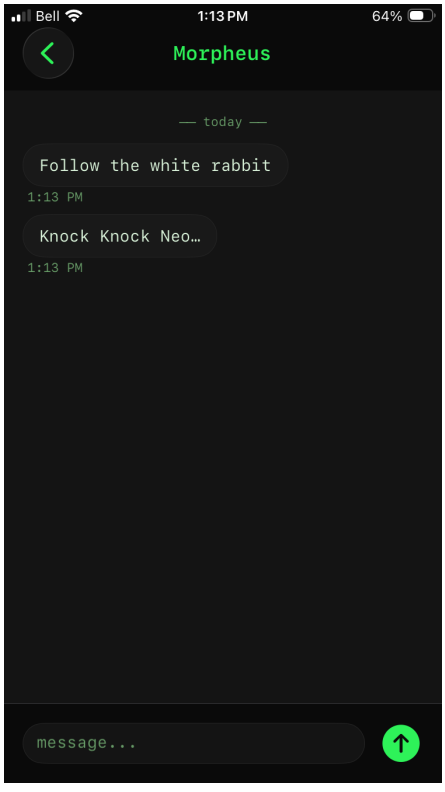
Annexe 3 — Visuels de l'application

Accueil

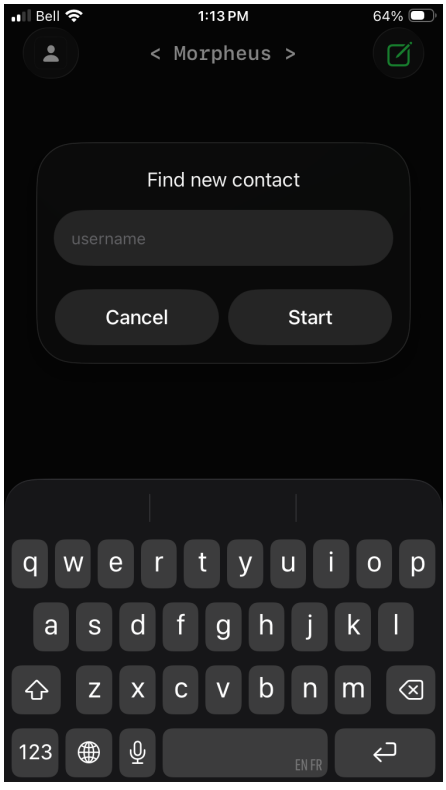
Choix de compte



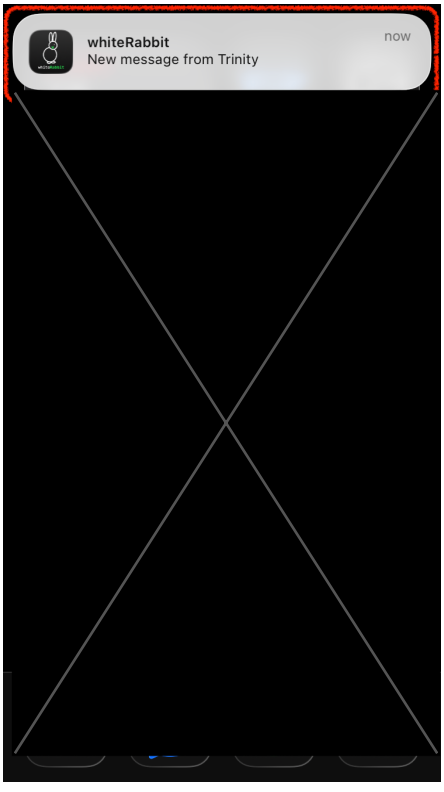
Messagerie



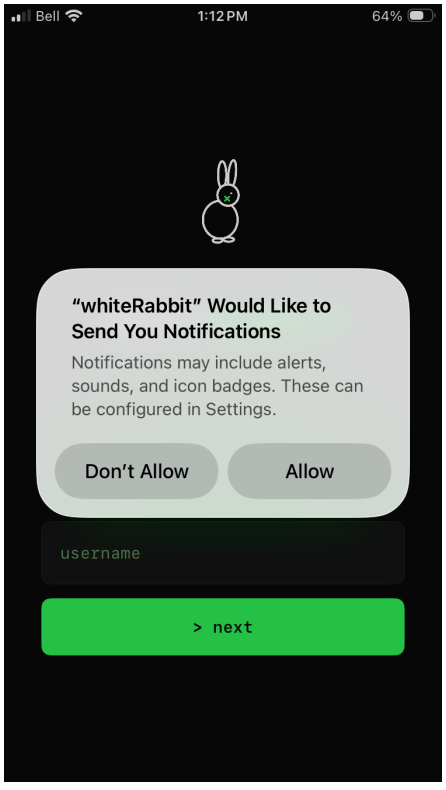
Recherche de contact



Notifications



Permissions



Références

[1] E. Snowden, *Mémoires vives*, trad. F. Cotton. Paris : Seuil, 2019.

[2] NIST (2016). Post-Quantum Cryptography Standardization. NIST.

[3] NIST (2024). FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard.

[4] NIST (2024). FIPS 204 — Module-Lattice-Based Digital Signature Standard.

[5] NIST (2024). FIPS 205 — Stateless Hash-Based Digital Signature Standard.

[6] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in Proc. 35th Annual Symposium on Foundations of Computer Science, 1994, pp. 124–134.

[7] Q. D. Truong, Y. Jang, and H. Lee, "High-Performance Unified Hardware Architecture for ML-DSA and ML-KEM PQC Standards," IEEE Access, vol. 13, pp. 189444–189460, 2025, doi: 10.1109/ACCESS.2025.3628733.

[8] M. Marlinspike and T. Perrin (2016). The Signal Protocol. Open Whisper Systems.

[9] M. Marlinspike and T. Perrin (2016). The X3DH Key Agreement Protocol. Signal.

[10] M. Marlinspike and T. Perrin (2016). The Double Ratchet Algorithm. Signal.

[11] E. Hir and J. Brendel (2023). PQXDH: Post-Quantum Extended Diffie-Hellman. Signal.

[12] Apple (2024). iMessage with PQ3: The new state of the art in quantum-secure messaging.

[13] L. Ibsen. SwiftKyber. GitHub. https://github.com/leif-ibsen/SwiftKyber

[14] L. Ibsen. SwiftDilithium. GitHub. https://github.com/leif-ibsen/SwiftDilithium

[15] Apple. CryptoKit. Apple Developer Documentation.

[16] Open Quantum Safe Project. liboqs. https://openquantumsafe.org

[17] Apple (2025). CryptoKit — ML-KEM support (iOS 26 preview).

[18] F. Turan and I. Verbauwhede, "Compact and Flexible FPGA Implementation of Ed25519 and X25519," ACM Transactions on Embedded Computing Systems (TECS), vol. 18, no. 3, pp. 1–21, 2019, doi: 10.1145/3312742.

[19] IEEE Computer Society LAN/MAN Standards Committee, IEEE standard for local and metropolitan area networks: media access control (MAC) security. Amendment 1, Galois counter mode–advanced encryption standard–256 (GCM-AES-256) cipher suite. New York: IEEE, 2011. [Online]. Available: http://ieeexplore.ieee.org/servlet/opac?punumber=6047534

[20] Gouvernement du Canada. Loi sur la protection des renseignements personnels et les documents électroniques (LPRPDE).

[21] Gouvernement du Québec (2021). Loi 25 — Loi modernisant des dispositions législatives en matière de protection des renseignements personnels.

[22] Gouvernement du Canada. Code criminel, Partie VI — Interception des communications privées.

[23] K. Cohn-Gordon et al. (2020). A Formal Security Analysis of the Signal Messaging Protocol.

[24] K. Hashimoto et al. (2022). A Concrete Treatment of Post-Quantum X3DH.

[25] Signal (2023). ML-KEM Braid specification.

[26] G. Greenwald and E. MacAskill (2013). NSA Prism program taps in to user data of Apple, Google and others. The Guardian.

[27] Parlement Européen (2016). Règlement Général sur la Protection des Données (RGPD).